

Local Public Interview: What You Actually Need

Systems Reliability Engineer | \$100k to \$110k | Remote | Start Nov 2

1. Read this first (2 minutes)

You cannot learn their tools in an hour. Do not try. That is not what wins this interview.

What wins it: show them (1) you have kept real systems running for real people, (2) you learn new tools fast, and (3) you are honest. Their tools are learnable in weeks. Judgment and ownership are not. That is your pitch.

2. The job in plain English

Local Public builds TV streaming apps (Roku, Apple TV, phones, Samsung and LG TVs, web) for 18 local PBS stations. Viewers watch shows and donate through those apps.

Your job: keep those apps and the computers behind them running and fast, notice problems before viewers do, release updates without breaking things, and help get new app versions into the app stores.

Why it matters to them: federal funding for public TV was cut in 2025. Stations now depend on viewer donations. Their apps raised station donations 53%. When the app is down, stations lose money.

Company facts: spun out of Cascade PBS (Seattle's PBS station) in 2026. CEO: Kevin Colligan. Goal: grow from 18 stations to every public TV station in 3 years. Small team.

3. Your 45-second intro (say it out loud 3 times)

"I'm an engineer who takes ownership of systems after they launch, not just before. At Apple I spent four years in the AI and machine learning group, and I was responsible for keeping a video system running in more than 40 labs in the US and China. I watched it, fixed it when it broke, and kept it reliable. Before that, I rebuilt a slow download system so a 24-minute job took 2 minutes and it handled five times the volume. Right now I run my own live website, career.ranklab.org, on cloud servers. I'm interested in this role because on a small team I'd own keeping your apps up, and your apps directly bring in donations for stations."

4. When they ask about tools you have not used (they will)

"Honestly, I haven't used some of your specific tools in production yet, like ECS or Terraform. What I have done is the actual job those tools serve: deploying code to many machines, keeping it running, and fixing it fast when it breaks. I've had to learn a new set of tools at every job, and I ramp up quickly. This year I learned Databricks and earned their certification in August. I'd expect to be comfortable with your setup within the first few weeks."

If they ask about a specific tool you don't know: **"I haven't used that one. How do you use it here?"** Then listen. Never fake it.

5. Words you will hear, translated

You don't need to explain these. You only need to not freeze when you hear them. The right column is what to connect it to.

Word	Plain meaning	What you've done that's similar
AWS / GCP	Amazon's and Google's rented computers in the cloud.	You used Amazon's cloud at Apple and Google's cloud at Lab 916.
Multi-cloud	They use both Amazon and Google at once.	You've used both.
Container (Docker)	An app packed in a box with everything it needs, so it runs the same anywhere.	At Apple you packaged Python code and shipped it to 40+ labs.
ECS	Amazon's service that runs those boxes and restarts them if they crash.	What you did by hand with SSH and scheduled jobs, but automated.
Serverless / Cloud Functions	Code that runs only when called. The cloud company handles the machines.	Your career.ranklab.org AI runs this way on Databricks.
CDN / CloudFront	Copies of videos and files stored close to viewers so they load fast.	Nothing direct. Say: "makes sense, it cuts load on the main servers."
Terraform / "infrastructure as code"	Writing your server setup in text files so it can be rebuilt with one command.	Like your scripts at Redstone, but for servers.
CI/CD / GitHub Actions / Cloud Build	A robot that tests and releases code automatically every time someone saves code to GitHub.	You use GitHub. You automated jobs at Redstone and Apple.
Monitoring and alerting	Dashboards plus automatic alarms when something breaks.	Your main Apple story.
Autoscaling	Adding more computers automatically when traffic jumps, removing them after.	Your capacity planning at Apple (predicting growth).
Uptime	Percent of time the app works.	Reliability of the Apple video system.
Rollback	Undo a bad update by going back to the last working version.	Common sense. Mention it in outage answers.
Postmortem	A written review after an outage: what broke, why, how to prevent it.	Say you'd write one after every incident.
On-call	Taking turns being the person who responds to alarms after hours.	Ask how they share it.

PostgreSQL, MongoDB, Aurora	Databases. Aurora is Amazon's database that grows and shrinks by itself.	You know SQL, MySQL, MongoDB.
Node.js, GraphQL	JavaScript on servers. GraphQL is a way for apps to ask the server for exactly the data they need.	You built APIs and know JavaScript.
Firebase	Google's toolkit for apps: logins, hosting, crash reports.	You've used it.
Latency	Delay. How long until something loads.	Your 24 min to 2 min story.

6. Likely questions, plain answers

"The app goes down tonight. What do you do?"

1. Find out how big it is: all stations or one? All devices or just Roku?
2. Ask what changed recently. Most outages come right after an update.
3. If an update caused it, undo it first. Fix properly later. Viewers come first.
4. Keep the team and stations updated while it's broken.
5. Afterwards, write up what happened and add an alarm or fix so it can't happen again.

"A big show airs and traffic jumps 10 times. How do you prepare?"

1. Add extra capacity before the show, not during.
2. Make sure videos are served from the copies near viewers (the CDN) so the main servers aren't flooded.
3. Test with fake traffic ahead of time.
4. Set alarms that fire early, before viewers notice.

"Tell me about a time something broke." Pick one real problem from Apple or Redstone before the call. Structure: what broke, how you found out, what you did, what you changed so it didn't happen again.

"Tell me about improving performance." The download system: it was fetching images one at a time. You made it fetch many at once. 24 minutes became 2, and it went from 100,000 to over 500,000 downloads a month.

"Are you OK with afternoon and evening hours?" "Yes. I'm in California on Pacific time, so your 9 to 1 core hours and evening coverage both work."

"Salary?" "The posted range of \$100k to \$110k works for me."

7. Questions for you to ask (pick 3)

1. "What does a typical week look like in this role?"
2. "What's been the biggest reliability problem in the last few months?"
3. "As you grow from 18 stations to 30 or more, what worries you most technically?"
4. "How is after-hours coverage shared on the team?"
5. "What would make someone successful here in the first 90 days?"

8. Last 5 minutes before the call

- Say your intro out loud once more.
- Have your resume and this sheet open.
- Water, quiet room, camera and mic tested.
- At the end ask: **"Is there anything about my background that gives you pause?"** Then answer it calmly.